

BBDD relacional para gestionar información en bootcamps de THB

Raul Rojo	DS
Alejandro Ruiz	DS
David Larreina	DS
Carlos Pmendiola	FS
Aldo Castillo	FS

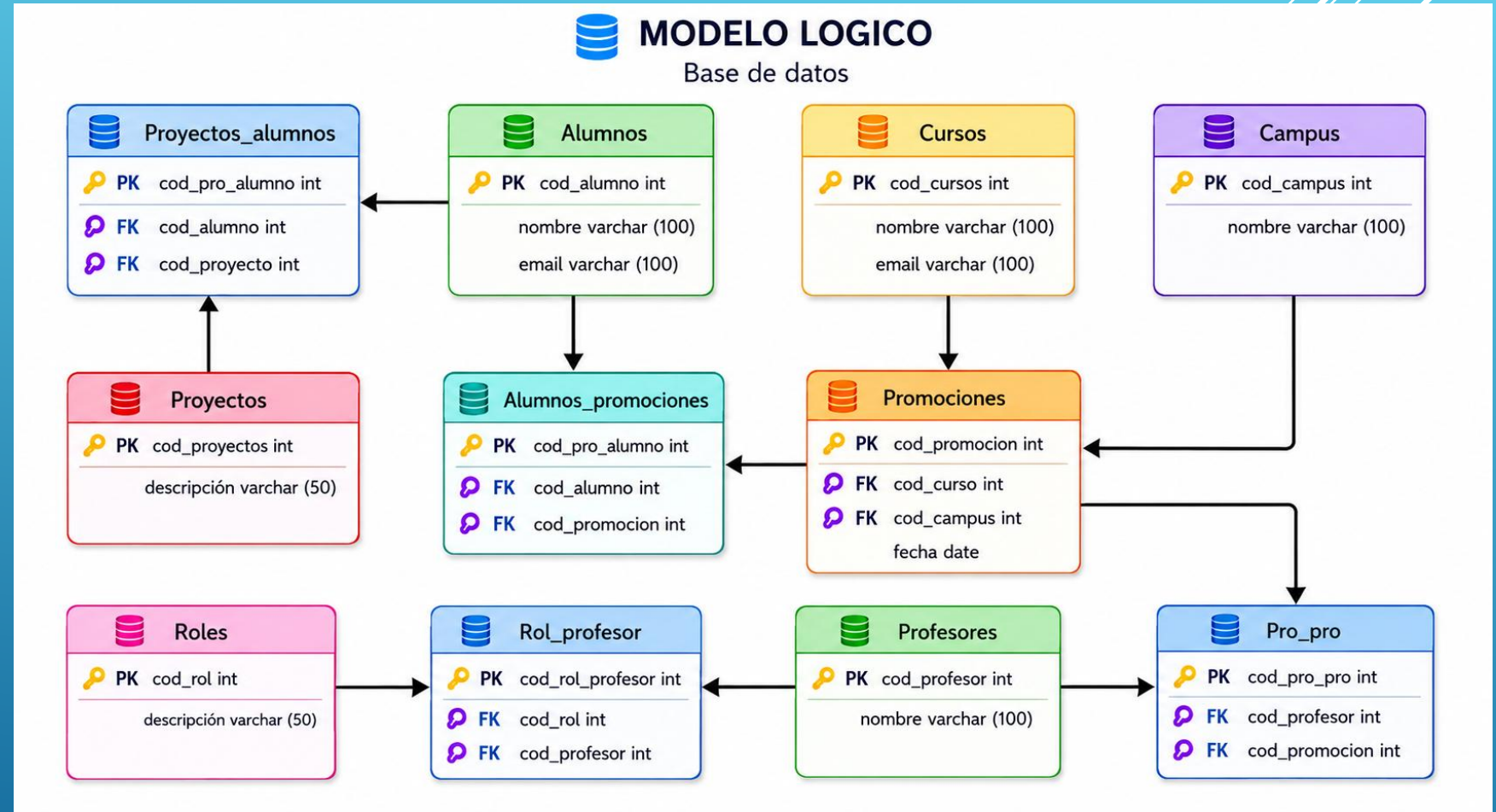


THE BRIDGE

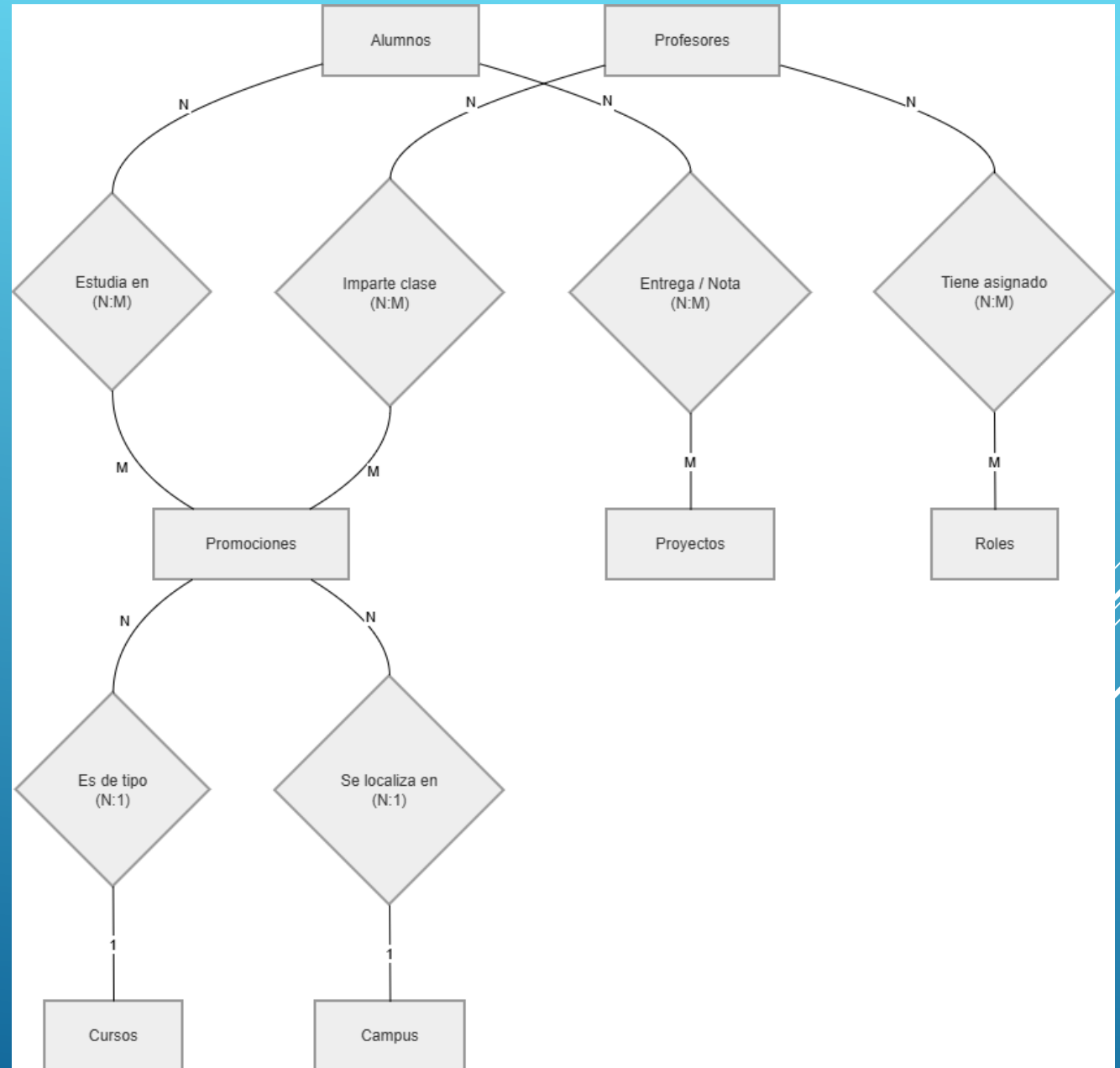
Modelo Lógico

Entidades:

- Proyectos_alumnos
- Proyectos
- Alumnos
- Alumnos_promociones
- Promociones
- Cursos
- Campus
- Pro_pro
- Profesores
- Rol_profesor
- Roles
- Rol_profesor



Modelo Entidad-Relación



Create Table

Query Query History

```
1 CREATE TABLE Alumnos (  
2   Cod_Alumno SERIAL PRIMARY KEY,  
3   Nombre VARCHAR(100),  
4   Email VARCHAR(100)  
5 );  
6 |
```

Query Query History

```
1 CREATE TABLE Proyectos_Alumnos (  
2   Cod_Proyecto_Alumno SERIAL PRIMARY KEY,  
3   Cod_Proyecto INT,  
4   Cod_Alumno INT,  
5   Nota BOOLEAN  
6 );|
```

Query Query History

```
1 CREATE TABLE Promociones (  
2   Cod_Promocion SERIAL PRIMARY KEY,  
3   Cod_Curso INT,  
4   Cod_Campus INT,  
5   Fecha DATE  
6 );|
```

Query Query History

```
1 CREATE TABLE Proyectos (  
2   Cod_Proyecto SERIAL PRIMARY KEY,  
3   Descripcion VARCHAR(50)  
4 );|
```

Alter Table

Query	Query History
1	-- PROYECTOS_ALUMNOS
2	
3	ALTER TABLE proyectos_alumnos
4	ADD CONSTRAINT fk_proyecto_alumno_alumno
5	FOREIGN KEY (cod_alumno)
6	REFERENCES alumnos(cod_alumno);
7	
8	ALTER TABLE proyectos_alumnos
9	ADD CONSTRAINT fk_proyecto_alumno_proyecto
10	FOREIGN KEY (cod_proyecto)
11	REFERENCES proyectos(cod_proyecto);

Query	Query History
1	-- ROL_PROFESOR
2	
3	ALTER TABLE rol_profesor
4	ADD CONSTRAINT fk_rol_profesor_rol
5	FOREIGN KEY (cod_rol)
6	REFERENCES roles(cod_rol);
7	
8	ALTER TABLE rol_profesor
9	ADD CONSTRAINT fk_rol_profesor_profesor
10	FOREIGN KEY (cod_profesor)
11	REFERENCES profesores(cod_profesor);

Limpieza de datos

```
1  -- datos Campus
2  INSERT INTO public.campus (nombre) VALUES
3  ('Madrid'),
4  ('Valencia');
5
6  -- Datos Cursos
7  INSERT INTO public.cursos (nombre) VALUES
8  ('Data Science'),
9  ('Full Stack');
10
11  datos Roles
12  INSERT INTO public.roles (descripcion) VALUES
13  ('TA'),
14  ('LI');
15
16  -- 1.4 Insertar Proyectos
17  INSERT INTO public.proyectos (cod_proyecto, descripcion) VALUES
18  (1, 'Proyecto_HLF'),
19  (2, 'Proyecto_BBDD'),
20  (3, 'Proyecto_ML'),
21  (4, 'Proyecto_FrontEnd'),
22  (5, 'Proyecto_Backend'),
23  (6, 'Proyecto_React'),
24  (7, 'Proyecto_FullStack'),
25  (8, 'Proyecto_EDA'),
26  (9, 'Proyecto_Deployment'),
27  (10, 'Proyecto_WebDev');
```

Inserte Into

```
Query  Query History
1  INSERT INTO public.promociones (cod_curso, cod_campus, fecha)
2  SELECT DISTINCT
3      CASE
4          WHEN "Promoción" ILIKE '%Data Science%' THEN 1
5          ELSE 2
6      END AS cod_curso,
7      CASE
8          WHEN "Campus" ILIKE '%Madrid%' THEN 1
9          ELSE 2
10     END AS cod_campus,
11     TO_DATE("Fecha_comienzo", 'DD/MM/YYYY') AS fecha
12 FROM (
13     SELECT "Promoción", "Campus", "Fecha_comienzo" FROM public.clase_1
14     UNION
15     SELECT "Promoción", "Campus", "Fecha_comienzo" FROM public.clase_2
16     UNION
17     SELECT "Promoción", "Campus", "Fecha_comienzo" FROM public.clase_3
18     UNION
19     SELECT "Promoción", "Campus", "Fecha_comienzo" FROM public.clase_4
20 ) AS temp_promos;
```


Inserte Into

```
Query  Query History
1  INSERT INTO public.alumno_promocion (cod_alumno, cod_promocion)
2  SELECT DISTINCT a.cod_alumno, p.cod_promocion
3  FROM (
4      SELECT "Email", "Promoción", "Campus", "Fecha_comienzo" FROM public.clase_1
5      UNION ALL
6      SELECT "Email", "Promoción", "Campus", "Fecha_comienzo" FROM public.clase_2
7      UNION ALL
8      SELECT "Email", "Promoción", "Campus", "Fecha_comienzo" FROM public.clase_3
9      UNION ALL
10     SELECT "Email", "Promoción", "Campus", "Fecha_comienzo" FROM public.clase_4
11 ) AS c
12 INNER JOIN public.alumnos a ON LOWER(TRIM(c."Email")) = LOWER(TRIM(a.email))
13 INNER JOIN public.promociones p ON
14     p.cod_curso = (CASE WHEN c."Promoción" ILIKE '%Data Science%' THEN 1 ELSE 2 END)
15     AND p.cod_campus = (CASE WHEN c."Campus" ILIKE '%Madrid%' THEN 1 ELSE 2 END)
16     AND p.fecha = TO_DATE(c."Fecha_comienzo", 'DD/MM/YYYY');
```


Por último, se realizaron una serie de consultas para comprobar que la base de datos funcionaba correctamente:

Query1: Alumnos que han aprobado al menos un proyecto.

Query2: Profesores que tienen asignado un rol específico.

Query3: Consultar promociones con información del curso y campus.

Query4: Consultar alumnos asociados a promociones.

Query5: Consultar alumnos con su curso y campus.

Query6: Consultar profesores con curso asignado.

Query7: Consultar proyectos asignados a alumnos.

Query8: Alumnos que han suspendido al menos un proyecto.

Query9: Promedio de proyectos aprobados por campus (rendimiento por ubicación).

Query10: Detectar alumnos con rendimiento irregular: aprobaron los tres proyectos iniciales, pero suspendieron alguno de los dos finales